

# Resumen Teorico – Practico 3

Diseno de tests basado en criterios

Particionado del Espacio de Entradas

*Testing de Software 2024*

## Índice

|                                                                                |          |
|--------------------------------------------------------------------------------|----------|
| <b>Introduccion</b>                                                            | <b>2</b> |
| <b>1. Marco general: Diseno de tests basado en criterios</b>                   | <b>2</b> |
| 1.1. El problema y la idea central . . . . .                                   | 2        |
| 1.2. Requisitos de test y criterios de cobertura . . . . .                     | 3        |
| 1.3. Flujo MDTD . . . . .                                                      | 3        |
| 1.4. Cuatro estructuras para modelar software . . . . .                        | 3        |
| 1.5. Cajas coloreadas y MDTD . . . . .                                         | 4        |
| 1.6. Requisitos infeasibles y grado de cobertura . . . . .                     | 4        |
| 1.7. Subsumption, conjuntos minimos y minimales . . . . .                      | 5        |
| 1.8. Generadores, reconocedores y herramientas . . . . .                       | 5        |
| 1.9. Por que conviene disenar tests basados en criterios . . . . .             | 6        |
| 1.10. Caracteristicas de un buen criterio de cobertura . . . . .               | 6        |
| <b>2. Particionado del Espacio de Entradas</b>                                 | <b>6</b> |
| 2.1. Motivacion . . . . .                                                      | 6        |
| 2.2. Particiones, propiedades formales . . . . .                               | 6        |
| 2.3. Particiones mal definidas: un caso instructivo . . . . .                  | 7        |
| 2.4. Modelado del dominio de las entradas: los 5 pasos . . . . .               | 7        |
| 2.5. Dos enfoques para construir el MDE . . . . .                              | 8        |
| 2.5.1. Enfoque basado en la interfaz . . . . .                                 | 8        |
| 2.5.2. Enfoque basado en la funcionalidad . . . . .                            | 8        |
| 2.5.3. Ejemplo: <code>kindOfTriangle(int side1, int side2, int side3)</code> . | 9        |
| 2.5.4. Ejemplo: <code>numberOfOccurrences(List, element)</code> . . . . .      | 9        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 2.6. Valores de test y estrategias . . . . .                    | 9         |
| <b>3. Criterios de cobertura sobre el espacio de entradas</b>   | <b>10</b> |
| 3.1. All Combinations Coverage (ACoC) . . . . .                 | 10        |
| 3.2. Each Choice Coverage (ECC) . . . . .                       | 10        |
| 3.3. Pair-Wise Coverage (PWC) y t-Wise Coverage (TWC) . . . . . | 11        |
| 3.4. Base Choice Coverage (BCC) . . . . .                       | 11        |
| 3.5. Restricciones entre características . . . . .              | 12        |
| 3.6. Comparacion rapida de criterios . . . . .                  | 12        |
| <b>4. Conexion con los ejercicios del TP3</b>                   | <b>13</b> |
| <b>5. Glosario rapido</b>                                       | <b>14</b> |
| <b>Cierre</b>                                                   | <b>14</b> |

# Introduccion

Este resumen sintetiza el material de teoria asociado al TP3:

- *Introduction to Software Testing* (Ammann & Offutt, 2da edicion), Capitulo 6: **Input Space Partitioning**.
- *Notas 05: Diseno de tests basado en criterios* (V. Bengolea).
- *Notas 06: Particionando el espacio de entrada* (V. Bengolea).

El objetivo del resumen es servir de “hoja de ruta” para repasar el practico: definiciones, intuiciones, ejemplos minimos y como se relacionan los conceptos entre si. La presentacion va de lo general (Model-Driven Test Design) a lo especifico (criterios sobre dominios de entrada como ECC, PWC, BCC y ACoC).

## 1. Marco general: Diseno de tests basado en criterios

### 1.1. El problema y la idea central

Para casi cualquier programa, el conjunto de entradas posibles es practicamente infinito. No podemos probar todas las entradas. La pregunta operacional del testing es:

*Dado un presupuesto finito de tests, ¿como elegirlos para maximizar la probabilidad de detectar fallas?*

El enfoque **Model-Driven Test Design (MDTD)** responde con una idea simple: *modelar* el software de alguna forma estructurada y *recorrer* ese modelo siguiendo reglas explicitas. Las reglas son los **criterios de cobertura**.

#### Idea clave

“La tarea del tester es definir un modelo del software y encontrar formas de recorrerlo.”

## 1.2. Requisitos de test y criterios de cobertura

### Definicion

**Requisito de test (TR).** Un elemento especifico del artefacto software (o de su modelo) que un test debe satisfacer o cubrir.

**Criterio de cobertura.** Una coleccion de reglas que impone los requisitos de test sobre el conjunto de tests.

**Cobertura.** Dado un conjunto de requisitos de test  $TR$  para un criterio  $C$ , un conjunto de tests  $T$  satisface  $C$  sii

$$\forall tr \in TR, \exists t \in T \mid t \text{ satisface } tr.$$

Un mismo modelo se puede recorrer con criterios distintos, y cada criterio produce su propio conjunto de TRs. Cambiar el criterio cambia el costo (numero de tests) y la rigurosidad de la suite.

## 1.3. Flujo MDTD

El flujo recomendado tiene dos niveles de abstraccion:

### 1. Nivel de diseno.

- a) Partir del artefacto software (codigo, especificacion, casos de uso, etc.) y construir un **modelo / estructura**.
- b) Aplicar un criterio para derivar **requisitos de test**.
- c) Refinarlos hasta tener **especificaciones de test** (que se va a probar y como).

### 2. Nivel de implementacion.

- d) Elegir **valores de inputs** para cada especificacion.
- e) Codificarlos como **scripts de test** ejecutables.
- f) Correr la suite y obtener **resultados** (pasa / falla).

La separacion entre diseno e implementacion es importante: las decisiones de modelado se toman a nivel de especificacion, y solo despues bajamos al codigo.

## 1.4. Cuatro estructuras para modelar software

El libro distingue cuatro formas de modelar el software, cada una con su propio espacio de criterios:

- **Espacio de inputs (conjuntos).** Caracterizar el dominio de las entradas y partitionarlo. *Es el tema central del Capitulo 6 y del TP3.*
- **Grafos.** Modelar flujo de control, flujo de datos, maquinas de estados, etc.
- **Logica.** Cubrir expresiones booleanas (condiciones en codigo, guardas, FNDs, FSMs).
- **Sintaxis.** Cubrir gramaticas (codigo, modelos, formatos de entrada, integracion).

## 1.5. Cajas coloreadas y MDTD

La terminologia clasica distingue:

- **Black-box testing:** derivar tests de descripciones externas al codigo (especificaciones, requisitos, diseno).
- **White-box testing:** derivar tests de la estructura interna del codigo (sentencias, condiciones).
- **Model-based testing:** derivar tests de un modelo abstracto (por ejemplo, un diagrama UML).

MDTD le resta importancia a esta clasificacion: lo que importa es *que modelo* se usa y *como* se recorre, no si la fuente es “externa” o “interna”.

## 1.6. Requisitos infeasibles y grado de cobertura

### Definicion

**Requisito de test inviable (infeasible).** Aquel para el cual no existen valores de entrada que lo satisfagan.

Tipico ejemplo: una rama de codigo que es *dead code*. La deteccion automatica de requisitos inviables es **indecidable** en general. Por eso casi nunca se alcanza el 100 % de cobertura en la practica.

### Definicion

**Grado de cobertura.** Porcentaje de los requisitos de test satisfechos por un conjunto de tests dado.

## 1.7. Subsumption, conjuntos minimos y minimales

### Definicion

**Subsumption.** El criterio  $C_1$  *subsume* a  $C_2$  si y solo si *todo* conjunto de tests que satisface  $C_1$  tambien satisface  $C_2$ .

Es una relacion entre criterios, no entre suites concretas. La intuicion es: “si paso  $C_1$ , gratis paso  $C_2$ ”. Hay dos formas tipicas de que esto suceda:

- Los TRs de  $C_1$  son un *superconjunto* de los de  $C_2$ .
- Existe una correspondencia muchos-a-uno entre TRs de  $C_1$  y de  $C_2$ . Si la correspondencia es uno-a-uno, los criterios se subsumen mutuamente.

### Ejemplo

*Caramelos.* Si tenemos 6 sabores agrupados en 4 colores, comer un caramelo de cada sabor implica haber comido al menos uno de cada color (porque cada color esta cubierto por algun sabor). El criterio “probar todos los sabores” **subsume** al criterio “probar todos los colores”.

En cambio,  $T_2 = \{\text{limon, pistacho, pera, mandarina}\}$  cumple el criterio de color (amarillo, verde, blanco, naranja) pero *no* cumple el de sabor (falta damasco y melon). Por lo tanto el criterio de color *no* subsume al de sabor.

### Definicion

**Conjunto de tests minimal.**  $T$  satisface  $TR$  y si se elimina cualquier test, deja de satisfacerlo.

**Conjunto de tests minimo.**  $T$  satisface  $TR$  y no existe otro  $T'$  mas chico que tambien lo satisfaga.

Todo conjunto minimo es minimal, pero no al revés: puede haber varios conjuntos minimales de tamanos distintos.

## 1.8. Generadores, reconocedores y herramientas

- **Generador:** procedimiento que produce valores que satisfacen un criterio. Es lo que tipicamente uno espera, pero suele ser dificil sin herramientas.
- **Reconocedor:** procedimiento que decide si un conjunto de tests dado satisface un criterio. Suele ser mas facil.

En la industria es comun escribir tests “a ojo” y luego *medir* la cobertura con un reconocedor (por ejemplo, JaCoCo para cobertura de codigo).

## 1.9. Por que conviene disenar tests basados en criterios

- Maximiza el beneficio del tiempo invertido en testing: menos tests que son mas efectivos.
- Produce suites amplias con minimo solapamiento.
- **Trazabilidad**: cada test esta justificado por uno o varios TRs. Esto facilita el testing de regresion y la revision de la suite.
- Da una **regla de parada** explicita: cuantos tests hace falta tener (o cuanta cobertura).

## 1.10. Caracteristicas de un buen criterio de cobertura

Un buen criterio:

1. Permite *calcular los TRs automaticamente* (o al menos de forma sistematica).
2. Permite *generar valores de test eficientemente*.
3. Produce suites con *buena capacidad de deteccion* de defectos.

# 2. Particionado del Espacio de Entradas

## 2.1. Motivacion

El **dominio de entradas** de un programa contiene todas las entradas posibles. Incluso para programas pequeños es practicamente infinito. La idea del particionado es:

### Idea clave

Dividir el dominio en **bloques** mediante **caracteristicas** relevantes para testing y elegir al menos un valor de cada bloque. Dentro de un mismo bloque, los valores se asumen “igualmente utiles” para detectar defectos.

## 2.2. Particiones, propiedades formales

Para un dominio  $D$ , una particion  $q$  define un conjunto de bloques  $B_q = \{b_1, b_2, \dots, b_n\}$  que debe satisfacer:

1. **Disjointness (no solapamiento)**.

$$b_i \cap b_j = \emptyset \quad \forall i \neq j, b_i, b_j \in B_q.$$

Ningun valor cae en dos bloques a la vez.

## 2. Completitud.

$$\bigcup_{b \in B_q} b = D.$$

Todo valor del dominio cae en algun bloque.

### Ejemplo

Para `abs(int n)` la característica natural es “relacion de  $n$  con cero”. Bloques:  $\{n = 0, n > 0, n < 0\}$ . Es completa y disjunta para `int`.

## 2.3. Particiones mal definidas: un caso instructivo

Un error tipico es elegir bloques que no son ni disjuntos ni completos. Considerese la característica “orden de un archivo  $F$ ” con bloques:

- $b_1$ : orden creciente,
- $b_2$ : orden decreciente,
- $b_3$ : orden arbitrario.

Si  $F$  tiene **longitud 1**, el archivo esta simultaneamente en los tres bloques (un elemento es “creciente”, “decreciente” y “arbitrario” a la vez). Hay solapamiento.

**Solucion:** dividir en dos características independientes, cada una con dos bloques:  $C_1$  = “orden creciente”  $\in \{\text{true}, \text{false}\}$ ,  $C_2$  = “orden decreciente”  $\in \{\text{true}, \text{false}\}$ . Ahora cada particion es disjunta y la combinacion de ambas describe correctamente las cuatro situaciones posibles.

### Idea clave

Regla practica: *una característica debe modelar una sola propiedad*. Si una particion parece “rara”, es mas seguro tener mas características con pocos bloques que pocas características con muchos bloques.

## 2.4. Modelado del dominio de las entradas: los 5 pasos

Modelar bien es la mitad del trabajo. El proceso canonico tiene 5 pasos:

### 1. Identificar la funcionalidad a testear.

- Testing de unidad: metodos individuales.
- Testing de integracion: metodos de una o varias clases.
- Testing de sistema: el programa completo.

## 2. Listar todas las entradas.

- Metodos: parametros y estado (variables no locales).
- Modulo: parametros y variables globales involucradas.
- Sistema: archivos, bases de datos, variables de entorno, etc.

## 3. Crear un modelo del dominio de las entradas (MDE).

- Identificar **caracteristicas** relevantes.
- Particionar cada caracteristica en **bloques**.
- Verificar completitud y disjointness.

## 4. Aplicar un criterio para generar combinaciones de bloques (ECC, PWC, BCC, ACoC).

## 5. Obtener entradas concretas a partir de las combinaciones (valores reales para ejecutar los tests).

Los pasos 1 y 2 estan en el nivel de abstraccion *concreto* (codigo real, parametros). Los pasos 3 y 4 se realizan en el nivel *de disenio* (modelo abstracto). El paso 5 vuelve al nivel concreto.

### Idea clave

Un MDE representa el espacio de entradas de un SUT (System Under Test) en forma abstracta. Permite razonar sobre que se prueba sin atarse a valores concretos.

## 2.5. Dos enfoques para construir el MDE

### 2.5.1. Enfoque basado en la interfaz

- Se generan caracteristicas a partir de cada parametro *aisladamente*.
- Es basado en sintaxis (tipos, presencia de `null`, etc.).
- Suele ignorar informacion semantica y las relaciones entre parametros.
- Es facil y mecanico, pero limitado.

### 2.5.2. Enfoque basado en la funcionalidad

- Las caracteristicas reflejan el *comportamiento esperado* del programa.
- Permite incorporar informacion semantica y del dominio.

- Captura relaciones entre parametros (por ejemplo, “ $a + b > c$ ” para un triangulo valido).
- Requiere mas esfuerzo de diseno.

#### Idea clave

**Heuristica de Ammann & Offutt:** las *precondiciones* y *poscondiciones* son fuentes excelentes de características. Separan explícitamente el comportamiento normal del excepcional.

#### 2.5.3. Ejemplo: `kindOfTriangle(int side1, int side2, int side3)`

**Enfoque por interfaz (refinado):** una característica por lado con bloques  $\{> 1, = 1, = 0, < 0\}$ . Hay  $4 \times 4 \times 4 = 64$  combinaciones posibles pero solo algunas corresponden a triángulos válidos.

**Enfoque por funcionalidad (semántico):** una característica única “clasificación geométrica” con bloques {escaleno, isosceles no equilatero, equilatero, invalido}. Notar que se ajusta el bloque “isosceles” a “isosceles no equilatero” para que sea *disjunto* de “equilatero”.

Una alternativa intermedia es usar varias características booleanas con *restricciones*:

- $q_1 = \text{escaleno}$ ,  $q_2 = \text{isosceles}$ ,  $q_3 = \text{equilatero}$ ,  $q_4 = \text{valido}$ .
- Restricciones:  $q_3 = \text{true} \Rightarrow q_2 = \text{true}$ ;  $q_4 = \text{false} \Rightarrow q_1 = q_2 = q_3 = \text{false}$ .

#### 2.5.4. Ejemplo: `numberOfOccurrences(List, element)`

- **Por interfaz:** dos características booleanas, una por parametro (“list es null”, “list es vacía”). No capta cuantas veces aparece el elemento.
- **Por funcionalidad:**
  - “Numero de ocurrencias”  $\in \{0, 1, > 1\}$ ,
  - “element esta primero en la lista”  $\in \{\text{true}, \text{false}\}$ ,
  - “element esta ultimo en la lista”  $\in \{\text{true}, \text{false}\}$ .

## 2.6. Valores de test y estrategias

Una vez fijadas las particiones, hay que elegir valores reales para cada bloque. Recomendaciones:

- Incluir valores que representen **uso normal**.

- Incluir **valores invalidos** y **valores especiales** (null, 0, cadenas vacias, true/false, espacios en blanco).
- Considerar **valores limite** (borders).
- Re-particionar bloques que sean “muy gruesos”.
- Balancear el numero de bloques entre características.
- Verificar disjointness y completitud antes de generar tests.

#### Ejemplo

Si la particion para “relacion con cero” es  $\{> 1, = 1, = 0, < 0\}$ , un buen conjunto de valores es  $\{2, 1, 0, -1\}$  en lugar de  $\{5, 1, 0, -5\}$ , porque incluye los **bordes** 1 y  $-1$ , que son los que disparan defectos tipicos (errores “off-by-one”).

### 3. Criterios de cobertura sobre el espacio de entradas

Una vez definidas las características y los bloques, hay que decidir como *combinarlos* para producir requisitos de test. Los criterios principales son:

#### 3.1. All Combinations Coverage (ACoC)

##### Definicion

Se generan todos los productos cartesianos posibles de bloques entre características. Si hay  $k$  características con  $n_1, \dots, n_k$  bloques cada una, el numero de tests es  $n_1 \times n_2 \times \dots \times n_k$ .

Es el criterio mas exhaustivo. En la practica casi nunca se usa porque explota combinatoricamente: para `kindOfTriangle` con tres características de 4 bloques son  $4 \times 4 \times 4 = 64$  tests, de los cuales solo unos pocos corresponden a triangulos validos.

#### 3.2. Each Choice Coverage (ECC)

##### Definicion

Cada bloque de cada característica debe aparecer en al menos un test.  
El numero de tests es igual al numero de bloques de la *característica con mas bloques*.

Es el criterio mas barato pero el menos exigente: no garantiza interacciones entre características. Suele detectar pocos defectos.

### Ejemplo

Para `kindOfTriangle` con tres características de 4 bloques cada una, alcanza con 4 tests: por ejemplo  $(2, 2, 2)$ ,  $(1, 1, 1)$ ,  $(0, 0, 0)$ ,  $(-1, -1, -1)$ . Cada bloque de cada característica aparece al menos una vez.

## 3.3. Pair-Wise Coverage (PWC) y t-Wise Coverage (TWC)

### Definición

**PWC.** Para cada par de bloques de características distintas, debe existir al menos un test que los cubra.

**TWC.** Generalización: cada tupla de  $t$  bloques de  $t$  características distintas debe estar cubierta. PWC es  $t = 2$ ; ACoC es  $t = k$  (número total de características).

El número de tests para PWC es *al menos* el producto de los bloques de las dos características con más bloques.

**Por que PWC es popular:** la evidencia empírica sugiere que la mayoría de los defectos en software se disparan por la interacción de a lo sumo dos parámetros. PWC ofrece una excelente relación costo/beneficio respecto a ACoC.

### Ejemplo

Tres particiones con bloques  $[A, B]$ ,  $[1, 2, 3]$  y  $[x, y]$  generan  $2 \times 3 + 2 \times 2 + 3 \times 2 = 16$  pares. Como cada test cubre 3 pares (uno por cada combinación de características), alcanzan 8 tests para cubrirlos todos:  $(A, 1, x)$ ,  $(A, 2, x)$ ,  $(A, 3, x)$ ,  $(B, 1, y)$ ,  $(B, 2, y)$ ,  $(B, 3, y)$ ,  $(A, 1, y)$ ,  $(B, 1, x)$ .

## 3.4. Base Choice Coverage (BCC)

### Definición

Para cada característica se designa un bloque como **elección base**. El test base se forma con la elección base de todas las características. Los demás tests se forman manteniendo todas las bases constantes excepto una, que toma cada uno de los bloques no base.

Número de tests:  $1 + \sum_i (n_i - 1)$ , donde  $n_i$  es el número de bloques de la característica  $i$ .

La intuición es: *fijar un escenario típico (camino feliz) y variar una característica a la vez*. Es muy útil cuando hay un caso “normal” claramente identificable y se quiere medir el impacto de cada variación.

### Idea clave

**El test base debe ser viable.** Tipicamente conviene elegir el “camino feliz” o el caso mas frecuente como base. La eleccion de las bases es una decision de diseno; conviene documentarla.

### Ejemplo

Para tres características A, B, C con 4 bloques cada una, eligiendo  $(A_1, B_1, C_1)$  como base, BCC produce  $1 + 3 + 3 + 3 = 10$  tests:  $(A_1, B_1, C_1), (A_2, B_1, C_1), (A_3, B_1, C_1), (A_4, B_1, C_1), (A_1, B_2, C_1), (A_1, B_3, C_1), (A_1, B_4, C_1), (A_1, B_1, C_2), (A_1, B_1, C_3), (A_1, B_1, C_4)$ .

Existe una variante llamada **Multiple Base Choice (MBCC)** en la que se eligen varias bases distintas y se repite el procedimiento para cada una.

## 3.5. Restricciones entre características

No todas las combinaciones de bloques son factibles. Las restricciones son de dos tipos:

- Un bloque de una característica *no puede* combinarse con un bloque específico de otra (combinación prohibida).
- Un bloque *solo puede* combinarse con un bloque específico de otra (combinación forzada).

Como se manejan segun el criterio:

- **ACoC y PWC:** se eliminan las combinaciones no viables del conjunto de TRs.
- **BCC:** si la variación produce una combinación no viable, se cambia uno de los valores base por otro no base hasta lograr una combinación factible.

### Ejemplo

Para `numberOfOccurrences` con características:

- A: ocurrencia del elemento  $\in \{\text{no aparece, una vez, mas de una}\}$ ,
- B: primero en la lista  $\in \{\text{true, false}\}$ ,
- C: ultimo en la lista  $\in \{\text{true, false}\}$ ,

hay dos restricciones obvias:

- $\neg(\text{no aparece} \wedge \text{primero} = \text{true})$ ,
- $\neg(\text{no aparece} \wedge \text{ultimo} = \text{true})$ .

Combinaciones invalidas:  $(A.B_1, B.B_1)$  y  $(A.B_1, C.B_1)$ .

## 3.6. Comparación rápida de criterios

| Criterio | Idea                                                      | # Tests                                                          | Comentario                                       |
|----------|-----------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------|
| ECC      | Cada bloque aparece al menos una vez.                     | $\max_i n_i$                                                     | Mas barato, menos efectivo.                      |
| PWC      | Cada par de bloques de características distintas aparece. | Al menos el producto de las dos características con mas bloques. | Buen balance costo/beneficio.                    |
| BCC      | Test base + una variación por bloque no base.             | $1 + \sum (n_i - 1)$ .                                           | Util cuando hay un “camino feliz” claro.         |
| ACoC     | Producto cartesiano completo.                             | $\prod n_i$ .                                                    | Maximo nivel de combinación, suele ser inviable. |

**Relacion de subsumption:** ACoC subsume a PWC, que subsume a ECC. BCC *no* se ordena linealmente con los anteriores: en general no subsume ni es subsumido por PWC, porque cubre escenarios distintos (variaciones controladas en lugar de pares forzados).

## 4. Conexion con los ejercicios del TP3

El TP3 instancia estos conceptos sobre cinco unidades concretas:

- **Ejercicio 2 – PWC sobre `numberOfCurrences`.** MDE basado en funcionalidad con cuatro características (referencias y forma de la lista). Las restricciones son centrales para que los TRs sean factibles.
- **Ejercicio 3 – analisis de un MDE + BCC sobre `intersection`.** El MDE original viola *completitud* y *disjointness* en la característica que describe la relacion entre los sets; el ejercicio pide encontrar contraejemplos y proponer un MDE corregido antes de aplicar BCC.
- **Ejercicio 4 – PWC sobre `PatternIndex`.** MDE combinado (interfaz + funcionalidad) con características para el resultado esperado. La suite revela dos defectos clasicos: precondiciones no validadas y manejo incorrecto de cadenas vacias.
- **Ejercicio 5 – PWC sobre `Iterator`.** Características booleanas que describen el estado del iterador. Como `Iterator` es una interfaz, hay que elegir implementaciones concretas para cada combinación (por ejemplo, usar `Collections.unmodifiableList` para forzar el caso “`remove()` no soportado”).

## 5. Glosario rapido

- **SUT:** *System Under Test*. La unidad/modulo/sistema que se prueba.
- **MDE:** Modelo del Dominio de Entradas (en ingles, IDM, *Input Domain Model*).
- **TR:** Test Requirement, requisito de test.
- **ECC:** *Each Choice Coverage*.
- **PWC:** *Pair-Wise Coverage*; caso particular de TWC con  $t = 2$ .
- **TWC:** *t-Wise Coverage*; generaliza PWC a tuplas de  $t$  características.
- **BCC:** *Base Choice Coverage*.
- **ACoC:** *All Combinations Coverage*.
- **MBCC:** *Multiple Base Choice Coverage*; variante con varias bases.
- **Subsumption:** relacion entre criterios,  $C_1$  subsume a  $C_2$  si toda suite que cumple  $C_1$  cumple  $C_2$ .
- **Infeasible TR:** requisito de test que ningun input puede satisfacer.

## Cierre

El particionado del espacio de entradas es una de las herramientas mas versatiles de testing moderno: *no requiere conocer la implementacion*, solo la especificacion del espacio de entradas; se adapta a unit, integration y system testing; y permite ajustar el esfuerzo cambiando de criterio. La diferencia entre una suite “casual” y una basada en criterios es exactamente la trazabilidad: cada test responde a un TR, y la suite completa se puede auditar y mantener con claridad.